

编译原理第八次作业参考答案

练习 5.1.1

考虑文法

$$\begin{aligned} S &\rightarrow E \mathbf{n} \\ E &\rightarrow E - T \mid T \\ T &\rightarrow T / F \mid F \\ F &\rightarrow (E) \mid \mathbf{digit} \end{aligned}$$

(1) 消除左递归

E 和 T 都是直接左递归的，按照标准方法消去后得到：

$$\begin{aligned} S &\rightarrow E \mathbf{n} \\ E &\rightarrow T E' \\ E' &\rightarrow - T E' \mid \varepsilon \\ T &\rightarrow F T' \\ T' &\rightarrow / F T' \mid \varepsilon \\ F &\rightarrow (E) \mid \mathbf{digit} \end{aligned}$$

(2) 语法制导定义

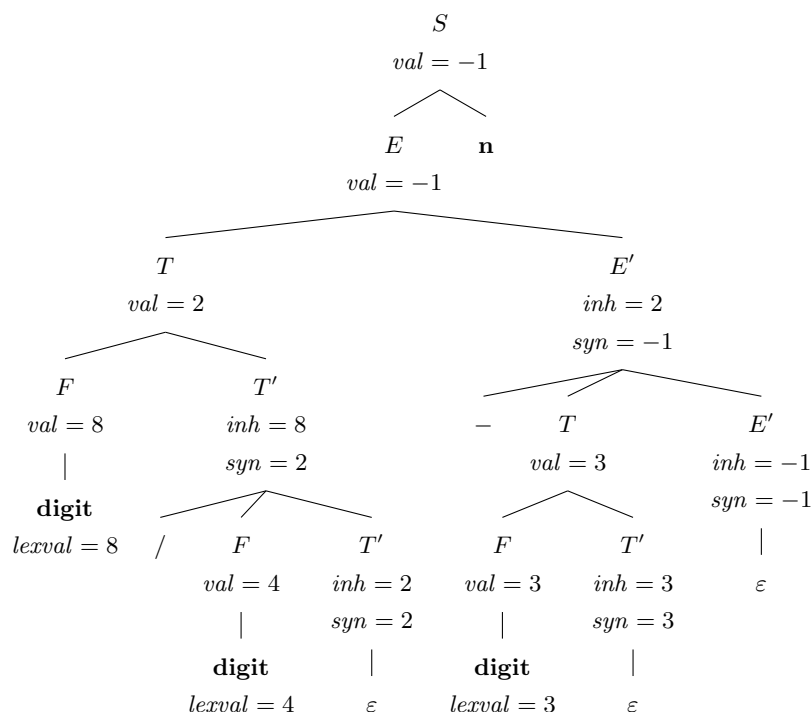
由于消除左递归后 E' 、 T' 的左部已经把“前缀的部分结果”作为继承属性向下传递，我们为 E' 和 T' 分别引入继承属性 inh （表示已经累积的左部值）和综合属性 syn （表示整个 E'/T' 子树计算完毕后的最终值）。其他非终结符使用综合属性 val 。SDD 如下：

产生式	语义规则
$S \rightarrow E \mathbf{n}$	$S.val = E.val$
$E \rightarrow T E'$	$E'.inh = T.val; \quad E.val = E'.syn$
$E' \rightarrow - T E'_1$	$E'_1.inh = E'.inh - T.val; \quad E'.syn = E'_1.syn$
$E' \rightarrow \varepsilon$	$E'.syn = E'.inh$
$T \rightarrow F T'$	$T'.inh = F.val; \quad T.val = T'.syn$
$T' \rightarrow / F T'_1$	$T'_1.inh = T'.inh / F.val; \quad T'.syn = T'_1.syn$
$T' \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

该 SDD 是 L-属性定义：每个继承属性只依赖于其在产生式中左侧符号的属性或父节点的继承属性。

(3) 输入 $8/4 - 3 \mathbf{n}$ 的注释语法分析树

记每个 E' 、 T' 结点旁的 (inh, syn) 表示其继承/综合属性，每个 F 、 T 、 E 结点旁标注 val ， S 结点标注最终的 val 。最终 $S.val = 8/4 - 3 = -1$ 。



练习 5.1.2

对产生式 $A \rightarrow B C D$ ，每个非终结符各有综合属性 s 与继承属性 i 。回忆定义：

- **S-属性定义**：所有属性都是综合属性（即语义规则中只出现自身或右部符号的综合属性，且头部 A 没有继承属性参与），等价地，规则中只为产生式左部设置综合属性。
- **L-属性定义**：对产生式右部第 j 个符号 X_j 的继承属性 $X_j.i$ ，其值只能依赖于：
 1. 产生式头部 A 的继承属性；
 2. $X_1, \dots, X_{j-1}$ （即在 X_j 左边的符号）的属性（综合或继承均可）；
 3. X_j 自身的继承属性，但需保证不产生循环依赖。

下面分别讨论三组规则。

(a) $A.s = B.s + C.s + D.s$

S-属性? 是。规则中只设置了产生式头部 A 的综合属性, 且只用到右部符号的综合属性, 符合 S-属性定义。

L-属性? 是。S-属性定义都是 L-属性定义的特例: 本规则没有为右部符号设置任何继承属性, 自然满足 L-属性的条件。

一致的求值过程? 存在。自底向上对每个右部子树先求值, 得到 $B.s, C.s, D.s$ 之后再计算 $A.s$ 即可, 对应一遍自底向上 LR 分析时的栈上求值。

(b) $B.i = A.i; C.i = B.s; D.i = B.s + C.s; A.s = B.s$

S-属性? 不是。规则为右部符号 B, C, D 都设置了继承属性, 已经超出 S-属性的范围。

L-属性? 是。逐条检查:

- $B.i = A.i$: B 是右部最左符号, 只用到了头部 A 的继承属性。✓
- $C.i = B.s$: B 在 C 的左边, 使用了左侧符号的综合属性。✓
- $D.i = B.s + C.s$: B, C 都在 D 的左边。✓
- $A.s = B.s$: 是头部 A 的综合属性, 由右部综合属性算出, 对 L-属性无额外限制。✓

故满足 L-属性。

一致的求值过程? 存在。按照 L-属性定义的标准做法:

1. 进入 A 子树时, 已知 $A.i$, 设置 $B.i = A.i$;
2. 递归计算 B 子树, 得到 $B.s$;
3. 设置 $C.i = B.s$, 递归计算 C 子树, 得到 $C.s$;
4. 设置 $D.i = B.s + C.s$, 递归计算 D 子树, 得到 $D.s$;
5. 计算 $A.s = B.s$ 。

该顺序即“从左到右一遍”, 可由递归下降的语法分析自然实现。

(c) $B.i = C.s$; $C.i = B.s + 1$; $A.s = B.s$

S-属性? 不是。同样为右部符号 B, C 设置了继承属性。

L-属性? 不是。 B 是右部最左符号, 但 $B.i = C.s$ 使用了右侧符号 C 的属性, 违反 L-属性定义中“ $X_j.i$ 只能依赖于左侧符号属性”这一要求。

一致的求值过程? 一般不存在。考察这条产生式在使用中的属性依赖:

- 通常 $B.s$ 由 B 的子产生式计算, 而其依赖会传递到 $B.i$ (即 $B.s$ 可能依赖 $B.i$);
- $C.s$ 同样可能依赖 $C.i$ 。

按本规则有

$$B.i = C.s, \quad C.i = B.s + 1.$$

合并后得到依赖链

$$B.i \longrightarrow C.s \longrightarrow C.i \longrightarrow B.s \longrightarrow B.i,$$

构成一个**循环依赖** (其中 $C.s$ 依赖 $C.i$ 、 $B.s$ 依赖 $B.i$ 这两个箭头来自 B 、 C 各自子树内部的常见依赖关系)。存在循环时, 没有任何属性能首先得到值, 因此不存在与之一致的求值过程。